D365

Teorija:

Caller(pokazivac) - referenca na objekat ili klasu koja inicira izvrsavanje neke druge operacije ili metode. Sa sobom nosi argumente koje postavimo(datum, comboBox, textBox).

Macro – nacin definisanja parametrizovanog skupa koda koji se moze koristiti vise puta. Korisni su jer slozene I cesto ponavljane delove koda definisemo samo jednom I koristimo gde god je potrebno. Promene u definiciji makroa automatski uticu na sve delove koda gde se koristi taj makro. Parametrizovan znaci da se moze koristiti odredjeni sablon koji se moze prilagoditi putem parametara(moze sadrzati delove koda koji se ponavljaju, ali mozemo uneti razlicite vrednosti ili argumente kako bismo prilagodili taj sablon za razlicite situacije).

```
str vehId;
str vehType;

#define.CurrentVersion(2)
#localmacro.CurrentList
    vehId,
    vehType
#endmacro
```

CheckFailed - default vrednost je false

```
ret = checkFailed("Please fill all mandatory fields");
```

This – se odnosi na trenutnu instancu forme ili klase. Iako se ovaj kod nalazi u formi ako unutar njega napisemo ‘this’ odnosice se na instancu klase AXMVehicleTable.

```
[DataSource]
class AXMVehicleTable
{
```

This je u tabeli konkretno taj red.

This u formi je klasa koja je najbliza. Ako si radio override onda se odnosi na tu override-ovanu.

Final – klasa koja ne moze da se nasledjuje

Display metoda – koriste se kako bi se prilagodile vrednosti koje se prikazuju korisnicima u interfejsu aplikacije kao sto su forma, izvestaji(report)... Omogucava razdvajanje logike prikaza od poslovne logike, sto cini kod citljivijim, odrzivim I fleksibilnijim. Mana je sto moze da se oteza testiranje I moze doci do povecanja slozenosti ako postoji veliki broj polja koji zahteva prilagodjen prikaz.

Caller() - referenca na ovu klasu. Ova klasa inicira pozivanje neke druge operacije ili metode.

```
AXMVehiclesListPage.xpp    TypeFilterCustDialog.xpp
TypeFilterCustDialog                                  ma

24
        1678 references
25      public void run()
26      {
27          Args args;
28          FormRun formRun;
29
30          args = new Args();
31          args.caller(this);
32          args.parm(operationField.value());
33          args.name(formStr(AXMVehiclesListPage));
34
35          formRun = classFactory.formRunClass(args);
36          formRun.init();
37          formRun.run();
38      }
```

```
AXMVehiclesListPage.xpp    TypeFilterCustDialog.xpp
DataSource:AXMVehicleTable                            executeQuery()

12      }
13
14      [DataSource]
15      class AXMVehicleTable
16      {
17          /// <summary>
18          ///  Executes query and decides filter type by comparing caller. If caller na
                TypeFilterCustDialog then filters data by type.
19          /// </summary>
20          public void executeQuery()
21          {
22              QueryBuildDataSource qbds;
23              qbds = this.query().dataSourceTable(tableNum(AXMVehicleTable));
24
25              if(element.args().caller())
26              {
27                  if(element.args().caller().name() == classStr(TypeFilterCustDialog))
28                  {
```

element.args().caller().name() - koristi se da bi se doslo do imena klase koja je pozvala odredjeni metod

element – odnosi se na trenutni element koji se obradjuje u kodu. U ovom slucaju AXMVehicleTable(jer vrednosti te tabele filtriramo). Ako bismo u init metodi napisali ‘element’ odnosilo bi se na formu.

Args() - argumenti prosledjeni objektu prilikom njegovog poziva

Caller() - omogucava pristup objektu (klasi) koji je pokrenuo trenutnu instancu. Referenca na objekat ili klasu koji inicira izvrsavanje neke druge operacije ili metode.

element.args().caller() - vraca referencu na klasu koja je pozvala metod unutar AXMVehicleTable

Name() - da dobijemo ime klase u obliku stringa

FormRun – koristi se za inicijalizaciju I upravljanje procesima vezanim za formu(otvaranje, zatvaranje, unos podataka I prikaz)

DataSource – pristupanje podacima u formi. Koristi se za upis, citanje, azuriranje, brisanje. Citanje podataka iz tabele. Na osnovu njega se radi query.

```
qbds = this.query().dataSourceTable(tableNum(AXMVehicleTable));
```

ClearRange() - brise trenutni filter

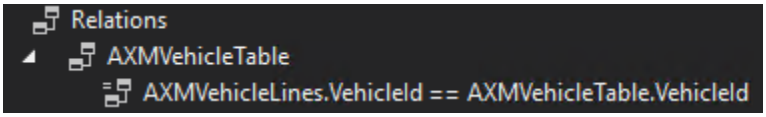
ClearRanges() - brise sve filtere query-ja. Koristi ga samo ako vec imas neki query inace ce ti pucati sa "Object reference not set to an instance of an object."

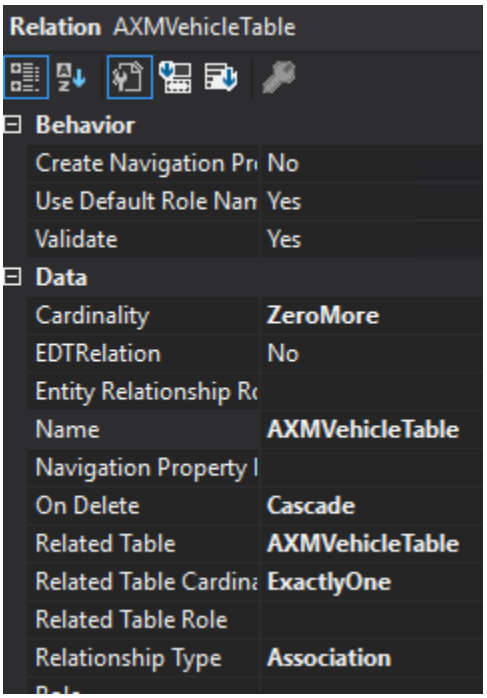
Dodavanje upita u query

```
qbds.addRange(fieldNum(AXMVehicleTable, VehicleType)).value(element.args().parm());
```

Kada odemo u systableData I obrisemo neki podatak, tj. Record obrisace se I na drugim mestima zato sto smo u tabeli na relaciji podesili kaskadno brisanje(property 'on delete')

https://usnconeboxax1aos.cloud.onebox.dynamics.com/?mi=systableBrowser&tablename=AXMVehicleLines&cmp=USMF





Parm() - pristupa se vrednosti parametara prosledjenim prilikom poziva metode. Cesto se prosledjuju kao argumenti prilikom poziva metode.

FormRun.design() - pristupanje dizajnu forme

Pristupamo forminoj kontroli koja se zove VehicleManufacturerTabPage I sakrivamo je

```
FormRun formRun = this.formRun();

FormTabPageControl tabPage =  formRun.design().controlName(formControlStr(EcoResProductDetails, VehicleManufacturerTabPage));

tabPage.visible(false);
```

Model – grupa elemenata, kao sto su obrasci(paterni), klase, tabele I bilo koji drugi objekat koji moze da se kreira u projektu. Modeli takodje mogu da sadrze dll-ove, reference na druge modele I metapodatke. Model ce na kraju biti deo onoga sto se zove paket, koji se moze koristiti za primenu koda u tom modelu u drugom okruzenju. Modelom se razliciti delovi organizuju u logicke celine, olaksavaju upravljanje verzijama, razliciti timovi mogu da rade na razlicitim modelima I omogucen im je istovremen razvoj bez konflikata, za promene mozes da se fokusiras samo na model koji sadrzi odredjene podatke, dodaci I prosirenja za D365FO mogu da se rade na nivou modela. Ako kreiras nov projekat I koristis svoj model neces poremetiti stari projekat jer ne mozes da menjas sve ono sto si radio.

Company USMF – nije bitno koja se kompanija koristi u aplikaciji. Nasa je bila ova iz nekog razloga, mozda jer ima korisnike.

Debagovanje(debugging) - attach to proccess, w3wp, F11. F11 je da ide korak po korak I ulazi u sve. F10 ne ulazi nego prolazi to sto je moje. Step out je da izadje iz toga gde je usao. Da bi mogao da debagujes COC moras da napravis ekstenziju onoga sto debagujes.

RunBase class – implementacija pozadinskih zadataka ili batch operacija. Ima dialog box, gde korisnik u unosi parametre pre pokretanja klase. Ima run, pack, unpack,… Serijalizacija se vrsi pomocu pack I unpack metoda. Serijalizacija omogucava da se poslednji izbor korisnika ponovo prikaze ako se dialog ponovo otvori. Takodje garantuje da podesavanja ostaju ista ako se klasa pokrece u pozadini.

RunBaseBatch class – objekat ove klase implementiraju sve potrebne metode koje sistem mora da ima da bi izvrsavao objekat kao batch job.

RunBase se izvrsava u dialogu, a RunBaseBatch u Batch-u. RunBase je osnovna klasa, a RunBaseBatch je podklasa(RunBaseBatch extends RunBase). U RunBase klasi imas opciju da izaberes da ces odraditi ‘run’ u batch-u.

Job – koncept koji se koristi za izvrsavanje zadataka u pozadini. To je kod koji se izvrsava asinhrono I automatski bez interakcije sa korisnikom.

Batch jobs – dozvoljava korisniku da izvrsava procese u pozadini. Uglavnom su to procesi kojima treba dosta vremena da se izvrse.

CanRunInNewSession(true) – da moze da pokrene nezavisno ili ako je false da je zavistan od nekih uslova okoline koje ne zelimo da delimo sa drugim sesijama

BatchProcessing – za obradu velikih kolicina podataka. Izvrsavaju se u grupama umesto pojedinacno.

Scheduled tasks – za izvrsavanje kada nema opterecenja sistema(na primer nocu)

Super – poziva kod koji nasledjujemo od roditelja da se izvrsi pa posle radimo funkcionalnosti. Kada se super pojavi u metodi odnosi se na logiku te metode. Znaci: super u create metodi poziva pozadinu za izvrsavanje create metode.

Super(append) - za dodavanje dodatnog ponasanja sa zadrzavanjem starog.

Next – isto kao super, odnosi se na logiku te metode koja se izvrsava u pozadini. Razlika je u tome sto se next koristi u chain of commands, a super kod override.
Ukoliko obrises 'next' nece moci da se kompajlira, a ukoliko obrises super moci ce.

```
public void initValue()
{
    super();
    this.TrainingType = TrainingType::Online;
}
```

```
public void insert()
    {
        this.Price = this.NoofDays * TrainingParameters::find().PerDayPrice;
        super();
    }
```

In the following example, the requirement is to initialize the **Price Rounding** rule of the **Currency** table with **0.01** whenever a new record is created. A table extension class will be created with this code snippet of the `initValue()` method.

X++ Copy

```
[ExtensionOf(tableStr(Currency))]
final class CurrencyTable_Extension
{
    public void initValue()
    {
        this.RoundOffPrice = 0.01;
        next initValue();
    }

}
```

While select nije uvek dobar jer prolazi mnogo record-a koji zadovoljavaju kriterijum.
Insert_recordset – pokupi sve podatke iz baze bez prolazenja 1 po 1. Ako bih imao milion podataka jako bi dugo trajalo, a ovako samo povuce sve.
Za sumiranje recimo mogu SUM() umesto da prolazim kroz while, to je optimalnije

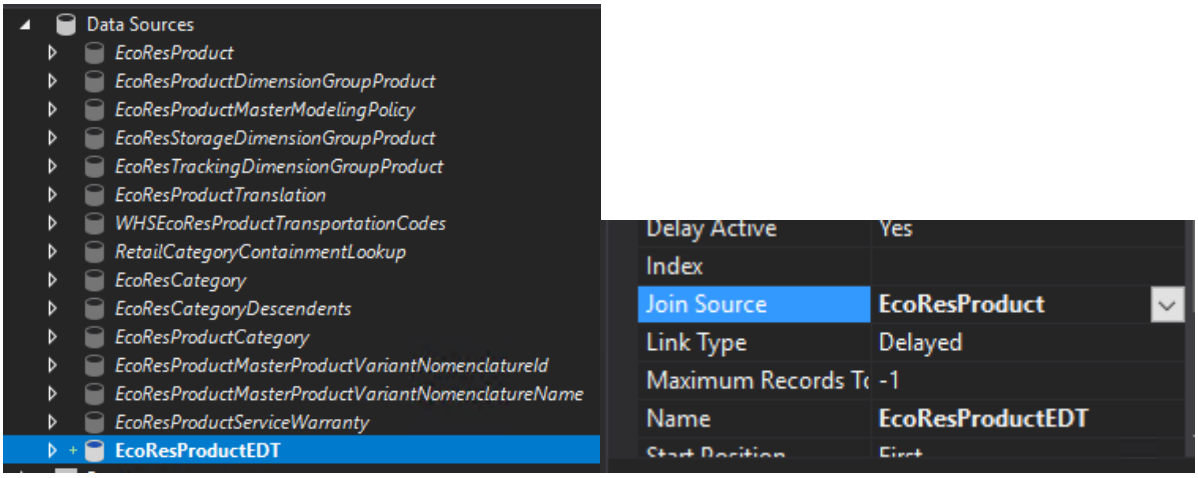
Chain of commands(COC) - koristi se da bi se izbeglo gomilanje slojeva. Pomocu njega pisemo logiku za metode pre ili posle njihovog izvrsavanja. Ako zelimo da pisemo COC u nekoj klasi ona mora biti final I njen naziv se zavrsava sa _Ectension. Radi se tamo gde override tj. Ekstenzije nisu moguce ili nisu dovoljne, a na ono sto nisi ti kucao I pravio. COC moze da radi prosirenje postojeceg EDT, metoda... Koristimo COC kada pravimo izmene na necemu sto nije nase(primer je task1). Ne mozes da menjas njihov kod pa odradis COC na tabelu.

Runnable klasa – klasa koju koristimo da napravimo update na nekim podacima u bazi

If-else - moze da se optimizuje tako sto napises if(!nesto). Ukoliko imas vise od 3 if-a onda koristis switch

If(neko polje == drugo neko polje) { nesto.enabled(true) else isto samo false – ne treba ovako Nesto.enabled(nekopolje == drugo polje)

Join source – kada zelimo da izvezemo vise entiteta u jednom izvozu. Moze da povlaci podatke, sve linije, sve producte i zato se linkuju dataSource-ovi. Recimo kada selektujes header I prikazuju se lines moze preko executeQuery, ali mogu dataSources da se povezu preko propertija sto je best practice.

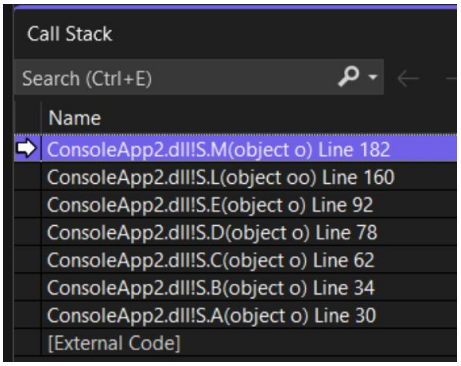


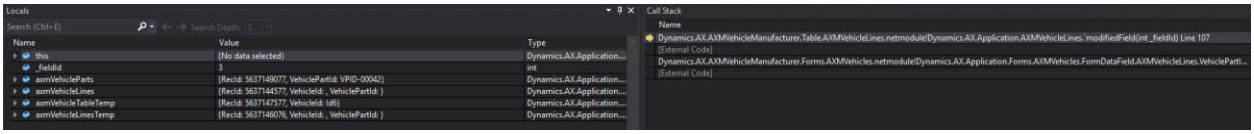
Callstack – deo debuggera koji pokazuje vise linija I svaka linija unazad je da istrazis odakle potice problem. Moze da se prati redosled poziva funkcija ili metoda tokom izvrsavanja programa. Obicno je organizovan kao stek podataka, gde se svaki novi poziv stavlja na vrh, a kada se funkcija zavrsi uklanja se. Koristi se za debagovanje, pracenje poziva funkcija, rekurzivne pozive, exception handling. Belezi metode koje prolazis.

Callstack se koristi da mozemo da vidimo gde je kod prosao pre nego sto zastane na nas breakpoint. Kliknemo na liniju I nosi nas na taj deo koda. Koristimo ga da pratimo tok aplikacije.

Callstack strelica – pokazuje gde se trenutno nalazi pokazivac izvrsenja. Informacije ovog steka se pojavljuju u locals, autos, watch I dissasembly window.

Locals – lokalne promenljive funkcije ili metode koja se izvrsava. Prikazuje vrednosti, tipove...
Autos – promenljive koje su relevantne za trenutni kod. Prikazuje promenljive trenutne linije I neke koje se cesto koriste
Watch window – rucno nadgledanje promenljivih ili izraza. Dodajes sta zelis da pratis I gledas kako se menjaju vrednosti
Disassembly window – pregledanje asemblerskog koda unutar procesa(za duboko razumevanje)





Query:

Predstavlja upit. U value postavljamo vrednost sa kojom uporedjuje.

```
[DataSource]
class AXMOrdersTable
{
    /// <summary>
    /// Query to show only records that have products inside of them
    /// </summary>
    public void executeQuery()
    {
        AXMOrderLines axmOrderLines;

        Query q;
        Queryrun qr;
        QueryBuildRange qbr;
        QueryBuildDataSource qbds;

        q = this.query();
        qbds = q.dataSourceTable(tableNum(AXMOrdersTable));
        qbds.clearRanges();
        qbr = qbds.addRange(fieldnum(AXMOrdersTable, OrderId));

        Container orderIds;
        while select axmOrderLines join axmOrdersTable
            group by axmOrderLines.OrderId
            where axmOrderLines.OrderId == axmOrdersTable.OrderId
        {
            orderIds += axmOrderLines.OrderId;
        }

        for (int i = 1; i <= conLen(orderIds); i++)
        {
            str orderId = conPeek(orderIds, i);

            qbr = qbds.addRange(fieldNum(AXMOrdersTable, OrderId));
            qbr.value(strFmt('(%1 == "%2")', fieldStr(AXMOrdersTable, OrderId), orderId));

            qr = new QueryRun(q);
        }
        super();
    }
```

Propertiji – property:

Insert if empty(na dataSource forme) – Na dataSource se postavlja na NO I ako je prazan line nece biti prikazano nista. Ukoliko ovde stoji YES kada otvorimo praznu formu bice unet prazan default red.

Allow duplicates – na indeks podesimo ovo I onda imamo ili nemamo duplikate

Tabele:

Table group property-ji:

Parameter– sadrzi staticke informacije o podesavanju modela kao sto su npr. Informacije o nacionalnoj valuti, metodi knjizenja... Postoji samo jedna instanca uzorka po kompaniji/modelu(tj. Ima samo jedan record).
WorksheetHeader – kategorizuje podatke u WorksheetLines. Odnos je one-to-many ka lines.
WorksheetLine – informacije koje treba validirati. Mozemo brisati linije bez uticanja na sistem.
Group – sadrzi podatke koji se primarno koriste za kategorizovanje jedne od glavnih tabela. Veza sa glavnom tabelom je one-to-many. Primer je VehicleFormula(za svaki tip automobila imam odredjene delove, a sa druge strane moze biti vise automobila nekog tipa). Isto I VehicleParts(jedan deo postoji, ali vise automobila moze da ima taj deo).

Imamo propertije:
Modified by, modified date time, created by, created date time
Replacement key – ovde ubacim kljuc koji se koristi za spajanje drugih tabela sa ovom. Koristimo ga da ispisemo smislenu vrednost na formi umesto numerickog primarnog kljuca.
Form Ref – ova forma se otvara klikom na neko polje kako bi se izvrsila detaljnija provera. Ako postavis 'Form Ref' za polje 'Customer' da referencira formu 'CustTable', kada korisnik klikne na polje 'Customer' u tabeli, otvorice se forma 'CustTable' sa detaljima o tom kupcu. Pravac od tabele ka formi.

Cache lookup – optimizovanje pretrage u kesu pre nego sto se pretrazi baza podataka. FoundAndEmpty znaci da ako postoje podaci u kesu koriste se oni I ne radi se upit u bazi. Kada se podaci iskoriste kes se prazni I time se obezbedjuje da ce se sledeci put odraditi upit ka bazi. Koristi se za brze odgovore, ali I da bi smo sa vremena na vreme izvrsili osvezili kes sa najnovijim podacima.

Field groups – grupisanje polja. Primer mesto stanovanja ima grad, ulicu, broj. Pored toga se koriste za laksi prikaz na formi. Prevucemo celu grupu umesto 1 po 1 polje.

Delete Actions

Mozemo da postavimo cascade umesto da ga postavljamo na relaciji

Propertiji polja:

Visible, Enum Type, EDT, Allow Edit, Mandatory

Da bi u sistemu mogao da dodam vise duplikata vrednosti potrebno je da odem na indeks I podesim property 'allow duplicates'

Indexes:

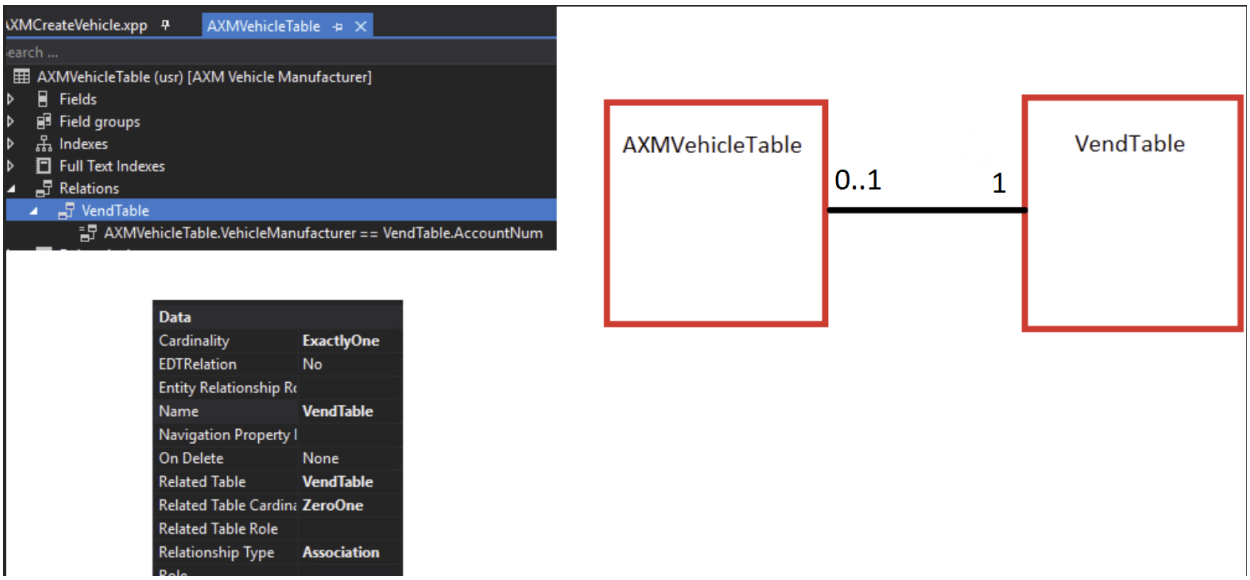
Ubrzanje pretrage I filtriranja podataka, ima ulogu I u sortiranju.
Alternate Key – da li je unique ili moze da se ponavlja

Relacije:

Moze biti Regular ili Foreign Key. U relaciji po poljima povezujemo razlicite tabele.

Foreign Key – sam stavlja(po defaultu) primarni kljuc kao jedno od polja, a ti samo dodas drugo

Cardinality – ZeroOne, ExactlyOne, ZeroMore, OneMore
Ovo je koliko se puta moze polje moje tabele pojaviti u drugoj



On Delete – None, Cascade, Restricted, CascadeRestricted
None - brisanje nije omoguceno
Cascade - brise podatke u svim povezanim tabelama gde je primarni kljuc ekvivalentan primarnom kljucu roditeljske tabele. Kada brisemo iz roditeljske tabele brise se i kod deteta.
Restricted - ako pokusas u roditeljskoj tabeli da obrises iskocice ti upozorenje ako taj resurs postoji kod deteta. Ukoliko brises preko X++ koda umesto user interfejsa mozes da obrises, ali samo kod deteta.
CascadeRestricted – ako obrises iz tabele ili interfejsa radice kao restricted, a ako obrises kroz kod radice kao cascade

Related Table – tabela sa kojom zelimo da spojimo nasu

Related Table Cardinality – ZeroOne, ExactlyOne
Koliko se puta polje povezane tabele(naziv relacije) moze pojaviti u mojoj tabeli

Relationship Type – Association, Composition, Link, Specialization, Aggregation
Asocijacija – postoji odnos izmedju dva entiteta, ali taj odnos nije strogo definisan. Ima strelicu na UML dijagramima. Prikazuje se opsti odnos, ali ne zelimo da definisemo detalje.
Kompozicija – entitet A se sastoji od entiteta B. Ukoliko jezero prestane da postoji nestaju I ribe.
Link – povezivanje entiteta, ali bez jake veze. Definise da li entiteti mogu biti povezani, ali bez preciznog definisanja tog odnosa. Manje formalno povezivanje gde nije potrebno duboko modeliranje odnosa.
Specijalizacija – kada jedan tip predstavlja poseban tip drugog. Za ‘Vozilo’ specijalizacija su ‘Auto’,

‘Kamion’, ‘Autobus’
Agregacija – entitet A sadrzi/agregira entitet. Ukoliko jezero nestane, patke mogu da odlete u drugo.

Forme:

Refresh forme:

FormDataSource formDataSource = FormDataUtil::getFormDataSource(this);

formDataSource.reread();

formDataSource.refresh();

FormDataUtil – staticka klasa za rad sa formama I njihovim podacima
U dataSource-u tabele na odredjeno polje radim override modified metode. U toj metodi radim refresh, ali sa 'reread'. Time ostajem na trenutnoj stranici u Dynamics-u. Ako stavim 'research' onda ce me vratiti na pocetnu vrednost.

ListPage – forma specijalizovana za pretrazivanje podataka zato se query koristi kao dataSource

DataSource:
Insert if Empty – da li se dodaje prazna linija ukoliko ne postoji ni jedna
Counter Field – LineNum brojanje linija, koristi se kao brojac ili indeks za zapise u toj formi, zbog toga se u create radi append. Pravimo jedinstven indeks za svaku liniju vozila I omogucavamo pracenje I identifikaciju

Refresh – radi refresh podataka koji se prikazuju na formi. Nece uraditi reread. Ukoli ne procitamo podatke ponovo, a promena se desila nece ih prikazati. **Ako nam nije bitno da li cemo zadrzati trenutne filtere. Bez obzira na njih ce izvrsiti ponovno ucitavanje.**

Reread – ima interakciju sa bazom podataka I pokrece query(upit) prema bazi da bi dohvatio nove/azurirane vrednosti zapisa.

Research – ponovo ucitava vrednosti iz baze podataka ili kesa. Ponovo pokrece postojeci upit(query) u bazi podataka I azurira liste novim/uklonjenim i radi update postojecih redova. **Ako zelimo da zadrzimo trenutne filtere I uslove. Radi upit zadrzavajuci sve postojece postavke.**

Research(true) - cuva poziciju kursora nakon sto se podaci osveze. Ako ne bismo imali 'true' prikazao bi samo najnoviji element, a stari se ne bi pojavili.

ExecuteQuery() - treba da se koristi ako si modifikovao upit I zelis prikaz na osnovu novog upita. Ukoliko je upit promenjen radimo 'ExecuteQuery' umesto 'Research'

formDataSource.queryRun().query() vs formDataSource.query()

FormDataSource.query() - originalna instanca dataSource query-ja.

FormDataSource.queryRun().query() - trenutna instanca query-ja koja se trenutno koristi sa nekim korisnickim filterima.

Kada korisnik pozove research kreira se nova instanca queryRun kao osnova. Ako je korisnik podesio neke filtere na prikazanim podacima oni ce biti sacuvani. Ovo je korisno kada vise korisnika radi sa odredjenim obrascima. Svaki ima svoje filtere podesene za prikazivanje samo relevantnih podataka, a

redovi se eksterno ubacuju u osnovnu tabelu.
Pozivanje executeQuery ce pozvati originalni query I ukloniti sve filtere.

```
[DataField]
class VehiclePartQty
{
    /// <summary>
    /// When vehicle part qty is modified we are edoing reread. Status is changed to open.
    /// </summary>
    public void modified()
    {
        super();

        FormDataSource fds = FormDataUtil::getFormDataSource(AXMVehicleTable);
        fds.reread();
        fds.refresh();
    }
}
```

List page – razlikuje se od ostalih formi jer ne moze da se doda dataSource nego mora query. Ideja je da kod ne moze da se pise na nivou forme(ali u D365 moze).

Dialog-Basic – za unos vrednosti novih parametara

Table of Contents – samo jedan tab I u njemu se dodaje ostalo: tab general, group, static text, general fast tab, tab page, groups

Propertiji:

Show New Button – na dizajnu forme

Dynamic fields(na dataSource) - da mozemo dinamicki da prilagodjavamo neka polja tj. Ukljucujemo/iskljucujemo. Programski mozemo da ih dodajemo I uklanjamo. Kada kliknemo ‘dynamic fields’ videcemo I base polja poput: RecId, Partition, RecStatus, CreatedBy, CreatedDateTime, ModifiedBy...

Security:

Ideja je da dobijes potpunu kontrolu

Role - sveobuhvatan posao

Duty – specifican task ili proces

Privilege – korak ili task procesa(duty procesa). U entry points prevlacis svoj menu item I podesavas ‘access level’. Ako stavis delete znaci da ce obuhvatiti I sve ostale, a ako stavis neki drugi obuhvatace njega I one iznad. Nivou su redom: No Access, Read, Update, Create, Correct, Delete.

Report:

Report – izvestaji su kolekcija grafikona I vizuelnih prikaza. Mozemo da ih koristimo da pratimo kljucne resurse I vidmo da li se resursi koriste efektivno

Ako imas otvoreno 2 reporta I pokusas da sacuvas nece moci jer ne zna gde da sacuva. Potrebno je da samo jedan ostane otvoren.

Svaki put kada napravis izmenu trebas da odradis 'deploy report'

Parm metoda u contract je zamena za get I set. Parm ujedno cini I get I set.

```
AXMVehicleTypeNone axmVehicleType;
[DataMemberAttribute('axmVehicleType'),SysOperationLabel('@AXM_VM:lbVehicleType')]
4 references
public AXMVehicleTypeNone parmAXMVehicleType(AXMVehicleTypeNone _axmVehicleType = axmVehicleType)
{
    axmVehicleType = _axmVehicleType;  set

    return axmVehicleType;             get
}
```

Temporary table za report se brise cim se prekine instanca. Ne sadrzi stvarne podatke, tj. nije popunjena. Moze biti TempDB I InMemory.

U tabelu mozemo prevuci polja iz drugih tabela.

Table type property:

TempDB – privremena baza koja se koristi za skladistenje privremenih podataka tokom sortiranja, spajanja, filtriranja I drugih operacija koje zahtevaju privremenu memoriju. Podaci se ne cuvaju trajno nego nestaju cim se prekine instanca.

InMemory – podaci se skladiste direktno u RAM. Omogucava brze I efikasnije pristupe jer se podaci nalaze u brzoj radnoj memoriji. Pogodno za slozene analiticke zadatke.

Kada napravimo report dodajemo report tabelu kao dataSource. U 'Parameters' node dodajemo polja koja ce se unositi tj. Koja ce se pojaviti u contract.

Design koji pravimo treba da bude 6.5 inch duzine jer se report sece na tu duzinu. Polja mozemo da grupisemo po nekom drugom polju. Da bismo obojili neka polja koristimo expression.

Set expression for: BackgroundColor

```
1 =IIF(Fields!VehiclePartPrice.Value > Parameters!
    AXMReportDS_thresholdPriceAmount.Value, "Red", "White")
```

Propertiji:
Dynamic filters – dinamicki filteri koji se primenjuju na report kako bi se omogucila analiza I filtriranje.
Data source type – 'Report Data Provider' je klasa koja se koristi za dobijanje podataka za izvestaje. Ukoliko postavimo 'AXM Enum Provider' koristimo enum, ali to se radi kada zelimo dobavljanje stanja, statuse umesto konkretnih reci I brojeva(omogucena bolja citljivost).

Propertiji na parametrima:
Allow Blank – moze da ostane prazno ili bez vrednosti bez izazivanje greske
Nullable – moze imati vrednost null tj. Vrednost nije poznata.

Grupisanje se u dizajnu reporta radi radi boljeg organizovanja, preglednosti, sumarizacije, filtriranja I usmeravanja paznje.

Za report mi je potrebno: report tabela, objekat reporta, constract, dataProvider I controller klase, ako imas query

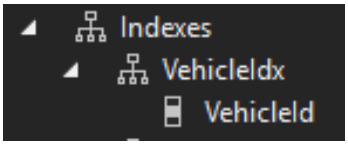
Tabele:

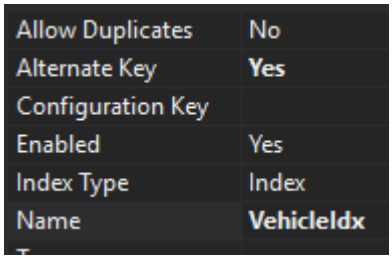
Auto Report – bar 2 polja moraju da se pojave u ovoj grupi. Polja koja su ovde postavljena ce oci da se stampaju kada korisnik klikne 'Print' u 'File menu'. Polja kandidati za ovu grupu su TitleField1 I TitleField2.

Auto Lookup – ako nista ne uneses u ovu grupu automatski ce sadrzati TitleField1, TitleField2 I polje koje je prvi unique index. Automatsko pretrazivanje I odabir vrednosti iz odredjenog tipa podataka. Ako je u ovoj grupi 'Kupac' kada korisnik pocne da kuca njegov naziv automatski ce se pretraziti moguci kupci tj. odraditi filtriranje. Korisno je za UX. Cesto se odnosi na kupce, dobavljace, proizvode gde je korisnicima potreban brz I precizan odabir.

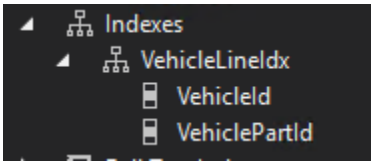
Indeks - pocinje od 1, ali ovde sugerise da je indeks usmeren na moje polje I ono se koristi za brzu pretragu. Indeksi se koriste jos I za: sortiranje, grupisanje(group by),...

Primarni kljuc mi treba da bih mogao da povezem ovu tabelu sa nekom drugom. Pravim relaciju table2.Polje1 == AXMVehicleTable.VehicleId





LineNum mi je kombinovani kljuc koji je nastao kombinacijom VehicleId I VehiclePartId. U tabeli moze da mi se pojavi isti LineNum ukoliko je u pitanju drugi VehicleId.



Metode:

Izvrsavaju se odmah: active, executeQuery,…

Init – inicijalizacija objekta pre nego sto se on koristi za rad tj. pre nego sto se prikaze korisniku. Automatski se poziva kada se objekat kreira. Koristi se za postavljanje pocetnih vrednosti, povezivanje sa podacima...

Run – izvrsavanje glavne logike. Obicno se poziva nakon sto su inicijalne vrednosti podesene u init metodi. Sadrzi kod za obradu I manipulaciju podacima, pozive drugih metoda I sve potrebno za izvrsavanje funkcionalnosti objekta

Active – proverava da li je polje omoguceno(enabled). Ako je omoguceno prikazuje ga, a ako nije ne prikazuje ga na formi. Radi kada si na dataSource forme.display

ModifiedField – ukoliko se promeni vrednost nekog polja. Kada imas modified na nivou tabele I dataSource-a izvrsava se prvo na dataSource. Mozes testirati tako sto stavis info na oba mesta, ali kod dataSource-a trebas da stavis iznad super() jer super poziva tabelu.

Create – kreira novi prazan red u dataSource forme I u njemu korisnik unosi podatke. Uglavnom se poziva na onClicked event. Aktivira se svaki put kada korisnik zeli da doda novi zapis(new).

JumpRef – koristi se da klikom na odredjeno polje odemo na drugu formu koja prikazuje specifican objekat. AXMVehicleTable je tabela cije je ovo polje, a AXMVehicles je forma na koju zelim da odem. Metoda je na nivou kontrole(polja tabele).

```
[Control("String")]
class VehicleManufacturer_VehicleId
{
    /// <summary>
    /// When specific vehicleId is pressed jumps to AXMVehicles form and shows this vehicle.
    /// </summary>
    public void jumpRef()
    {
        Args args = new Args();
        FormRun formRun;

        AXMVehicleTable axmVehiclesCursor = AXMVehicleTable_ds.cursor();

        // Specify the target form and pass the selected vehicleId as a parameter
        args = new Args(formStr(AXMVehicles));
        args.parm(axmVehiclesCursor.VehicleId);

        formRun = classfactory.formRunClass(args);
        formRun.init();
        formRun.run();
    }

}
```

Ekvivalentno SQL upitu: select * from AXMVehicleTable where VehicleId = ‘nekaVrednost’;

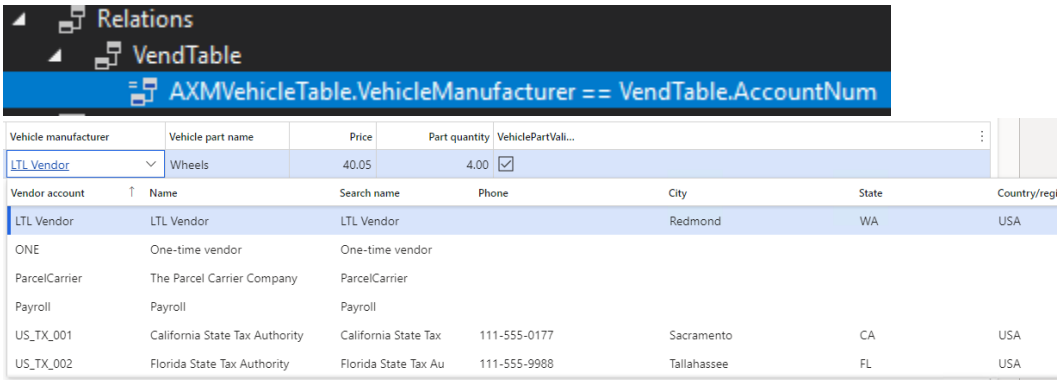
```
/// <summary>
/// Used for jumpRef to show lines for specific vehicle.
/// </summary>
public void init()
{
    super();

    // jumpRef part
    //Retrieve the parameter
    str axmVehicleTableCursor = element.args().parm();
    if (axmVehicleTableCursor)
    {
        // Clear dynamic links
        AXMVehicleTable_ds.query().dataSourceTable(tableNum(AXMVehicleTable)).clearDynalinks();

        // Add a range to filter by vehicleId
        AXMVehicleTable_ds.query().dataSourceTable(tableNum(AXMVehicleTable)).addRange(fieldNum(AXMVehicleTable, VehicleId)).value(queryValue
          (axmVehicleTableCursor));
        AXMVehicleTable_ds.executeQuery();
    }
}
```

Lookup – omogucava korisnicima da biraju iz odredjenog skupa podataka. Pravimo relaciju ili uslov koji odredjuje izmedju kojih vrednosti ce korisnik moci da izabere. Uslov je ustvari filter koji smanjuje skup podataka, a moze postojati I unos kljucnih reci a suzavanje rezultata.

Lookup – kada zelis da ti u tabeli prikaze vrednosti polja druge tabele pravis relaciju I lookup ce raditi automatski.



Ako zelis da povezes sa nekim drugim poljem gde nije moguce napraviti direktnu relaciju pravi se lookup metoda na polje u grid-u gde su kontrole ili ukoliko ces vise puta pozivati to polje u razlicitim gridovima pravi se na nivou dataSource-a.

Ako ne obrises super(); u lookup metodi desice ti se ovaj problem u Dynamics-u ukoliko vec imas lookup preko relacije.

More than one form was opened at once for the lookup control.

Kada napisem AXMVehicleTable on vidi 1 record koji je selektovan.

Ako napisem AXMVehicleTable_ds to je ceo dataSource.

_ds – odnosi se na sve podatke odredjene tabele.

AXMVehicleTable_q vidi query na nivou tog dataSource-a.

```
public void lookup(FormControl _formControl, str _filterStr)
{
    Query query = new Query();

    SysTableLookup sysTableLookup = SysTableLookup::newParameters(tableNum(AXMVehicleParts), _formControl);
    sysTableLookup.addLookupfield(fieldNum(AXMVehicleParts, VehiclePart));
    sysTableLookup.addLookupfield(fieldNum(AXMVehicleParts, VehiclePartId));

    query.addDataSource(tableNum(AXMVehicleParts)).addRange(fieldNum(AXMVehicleParts, VehicleType)).value(SysQuery::value
      (AXMVehicleTable.VehicleType));

    sysTableLookup.parmQuery(query);

    sysTableLookup.performFormLookup();
}
```

Edit metoda – na primer ukoliko zelis da umesto VEH-00072 vidis Fiat.

Pack – serijalizacija podataka iz objekta u niz bajtova. Konvertuje podatke iz objekta u niz bajtova koji se moze kasnije koristiti.

Unpack – deserijalizacija tj. Pretvaranje iz niza bajtova u objekat. Rekonstruise odgovarajuci niz u konkretan objekat.

Logika mesta odabira override metode:
Override na nivou tabele – bice u svakoj formi
Override na nivou dataSource – bice na nivou samo te forme
Override kontrole – bice samo na nivou te kontrole

Pristup trenutnom record-u:

U Event Handleru/formi:

axmVehicleTable.cursor();

axmVehicleTable_ds.cursor();

U klasi:

axmVehicleTable = args.record();

EDT:

EDT koristimo za razdvajanje logike I podataka. Mozemo postaviti validacije kao sto su duzina stringa, nenegativni brojevi, samo brojevi... Dodatno, mozemo staviti labele na EDT I prenosice se na sva polja koja taj EDT koristi(dodamo detaljniji opis za nasu organizaciju radi lakseg razumevanja). Mozemo promeniti vrednost kako nama odgovara, a ne da je NoYes(musko/zensko na primer). Takodje, mozemo koristiti I druge jezike, a ne samo jedan.

EDT VehicleId – pravim kako bih ovo polje mogao da koristim kasnije u kodu. Mozemo da dodajemo validacije, propertije za duzinu, broj decimala, a za number sequence moramo da korisitmo polje koje je ima EDT.

Za gresku sa migracijom na EDT – Ignore EDT relation(u polju), ali se poremete relacije tako da ne stavljaj YES za ovaj property.

Propertiji:

Extends – Date1980 za klasican prikaz datuma

Allow negative – ne moze da se unese negativna vrednost kod tipova real, int. Radi se u EDT(extended data type)

String size – kod EDT tipa string

Database String Size – kod EDT string

Menu and Menu items:

Menu – da bismo mogli nesto da prikazemo u dynamics meniju ekstendujemo MainMenu. U njemu dodamo menu reference I postavimo ime naseg menija. U nasem meniju pravimo submenije u koje prevlacimo nase menu item-e.

Menu itemi treba da imaju podesene propertije ‘object type’ I ‘object’ (koji tip itema gadjamo I kako se zove sam item)

Action menu item – za klase

Display menu item – otvaranje forme

Output menu item – pokretanje report-a

Kod - Code

Provera da li je decimalan ili ceo broj:

```
if(this.VehiclePartQty != real2int(this.VehiclePartQty))
{
    warning("@AXM_VM:messVehiclePartQuantityCantBeDecimalNumber");
    ret = false;
}
```

Number sequence – koristi se za generisanje citljivih, jedinstvenih identifikatora za maticne zapise podataka I transakcijske zapise koji zahtevaju identifikatore. Zapis glavnih podataka ili zapis transakcije koji zahteva identifikator naziva se referenca. Mi podesimo format koji zelimo da number sequence ima za neku kompaniju.

Da bi se odradio za pocetak se radi extend base enum klase 'NumberSeqModule' I dodamo polje koje se zove kao nas model.

Da bi se kreirao number sequence potrebna je 1 klasa.

Poziva se I definise parametre. Govorimo koji je naziv EDT-a, tekst koji ce se pojaviti u dynamicsu-u. Ostali parametri su standardni. Kada se pozove kreira se number sequence reference. Mi kreiramo 1 number sequence za kompaniju USMF, a moze da se kreira I po tabeli.

ParmWizardIsContinuous – da li ce brojcana serija biti kontinuirana
ParmWizardIsManual – da li su uneti rucno ili automatski generisani
ParmWizardFetchAheadQty – koliko brojeva se unapred generise
ParmWizardIsChangeDownAllowed – da li je dozvoljeno menjanje brojeva na dole
ParmWizardIsChangeUPAllowed – da li je dozvoljeno menjanje brojeva ka gore
ParmSortField(1) - znaci da ce se brojevi sortirati po prvom broju
DataArea – brojevi se razdvajaju prema DataArea vrednostima. True I false govore o omogucavanju I neomogucavanju ovoga

```
protected void loadModule()
{
    NumberSeqDatatype datatype = NumberSeqDatatype::construct();

    datatype.parmDatatypeId(extendedTypeNum(AXMVehiclePartId));
    datatype.parmReferenceHelp(literalStr("@AXM_VM:rtNSVehiclePartId"));
    datatype.parmWizardIsContinuous(false);
    datatype.parmWizardIsManual(NoYes::No);
    datatype.parmWizardFetchAheadQty(10);
    datatype.parmWizardIsChangeDownAllowed(NoYes::No);
    datatype.parmWizardIsChangeUpAllowed(NoYes::No);
    datatype.parmSortField(1);

    datatype.addParameterType(NumberSeqParameterType::DataArea, true, false);
    this.create(datatype);
}
```

Moj modul za koji kreiram number sequence. Number sequence ce se pojaviti u ovom modulu.

```
public NumberSeqModule numberSeqModule()
{
    return NumberSeqModule::AXMVehicleManufacturer;
}
```

Povezuje nasu klasu sa number sequence-om. ‘AddModuleMap’ se koristi za dodavanje odredjenog modula za numericke sekvence. ‘numberSeqModuleNamesMap’ dodaje informacije o modulu (AXMNumSeqVP) u tu mapu.

```
[SubscribesTo(classstr(NumberSeqGlobal),delegatestr(NumberSeqGlobal,buildModulesMapDelegate))]
1 reference
static void buildModulesMapSubsciber(Map numberSeqModuleNamesMap)
{
    NumberSeqGlobal::addModuleToMap(classnum(AXMNumSeqVP), numberSeqModuleNamesMap);
}
```

U dynamics-u kucam ‘Number Sequences’(organization administration > number sequences)

U Area stavljam ‘AXM Vehicle Manufacturer’ I onda ce mi se u References pojaviti da odaberem za sta kreiram, potom za company stavljam USMF. Kada udjem u Segments tabu podesavam format. Value=’VEH’, Constrant=’-’ , Alphanumeric=’#####’

Imam formu koja je dialog I kao dataSource je AXMVehicleTable. U tom dataSource-u pravim metodu create koja ce pozivati number sequence I postaviti mi njegovu vrednost kao ID polje.

```
[DataSource]
class AXMVehicleTable
{
    /// <summary>
    /// Shows number sequence when create is called.
    /// </summary>
    /// <param name = "_append">Does not append.</param>
    public void create(boolean _append = false)
    {
        AXMVehicleId axmVehicleId;
        NumberSequenceReference numberSequenceReference;
        numberSequenceReference = NumberSeqReference::findReference(extendedTypeNum(AXMVehicleId));

        super(_append);

        if (numberSequenceReference)
        {
            axmVehicleId = NumberSeq::newGetNum(numberSequenceReference).num();
            AXMVehicleTable.VehicleId = axmVehicleId;
        }
    }

}
```

Refresh the calling form – kada na dialogu kliknem OK radi se refresh forme za koju sam dodavao record u dialogu I odmah ga prikazuje.

Element – instanca dialog forme

Caller – ona forma na kojoj sam kliknuo New I otvorio ovaj dialog

FormRun – instanca forme koja je otorila dialog

Dobra praksa je provera da li je dsList == null iako bi uslov verovatno ispao kada bi se null>0 uporedilo. Verovatno se ne bi desio exception.

```
public void closeOk()
{
    FormRun formRun;
    List dsList;
    ListEnumerator dsListEnumerator;
    FormDataSource formDS;

    super();

    // Get an instance of the calling form.
    formRun = element.args().caller();

    // If the caller is a form, find and refresh the specified root data source.
    if(formRun)
    {
        dsList = formRun.rootFormDataSources();

        if(dsList && dsList.elements() > 0)
        {
            dsListEnumerator = dsList.getEnumerator();

            while(dsListEnumerator.moveNext())
            {
                formDS = dsListEnumerator.current();
                if(formDS.table() == tableNum(AXMVehicleTable))
                {
                    formDS.research(true);
                    break;
                }
            }
        }
    }
}
```

Prosledjuje se referenca na formu

```
args = new Args(formStr(AXMVehicleTypeParts));
```

Validacija imejla I broja telefona. Imejl moze da se ponovi osim ako nije primarni mejl koji je vec zauzet drugim korisnikom. Radjen je COC jer sve ovo vec postoji I dodatno se menja. Nalazi se u Modules > Accounts receivable > All customers

System.Text.RegularExpressions - namespace koji sadrzi ove dve klase

Regex – klasa koja dolazi iz .NET framework-a I predstavlja API za rad sa regularnim izrazima. Koristi se za kompilaciju I manipulaciju sa regularnim izrazima. Koristi se za definisanje sablona.

Match – klasa koja predstavlja rezultat pronalaska odgovarajuceg teksta u odredjenom izrazu. Match objekat sadrzi podatke o pronadjenim podudarnostima.

```
[ExtensionOf(tableStr(LogisticsElectronicAddress))]
final class LogisticsElectronicAddress_Table_Extension
{
    0 references
    public boolean validateField(FieldId _fieldIdToCheck)
    {
        boolean isValid = next ValidateField(_fieldIdToCheck);

        LogisticsElectronicAddress logisticsElectronicAddress;
        System.Text.RegularExpressions.Regex regex;
        System.Text.RegularExpressions.Match match;
        boolean regexMatchReturn;
        boolean email;

        if(isValid && (_fieldIdToCheck == fieldNum(LogisticsElectronicAddress, Locator)
                    || _fieldIdToCheck == fieldNum(LogisticsElectronicAddress, IsPrimary)))
        {
            if(this.Type == LogisticsElectronicAddressMethodType::Email
                || this.Type == LogisticsElectronicAddressMethodType::Phone)
            {
                if(this.Type == LogisticsElectronicAddressMethodType::Email)
                {
                    regex = new System.Text.RegularExpressions.Regex(@"^(\w+\@[a-zA-Z]+\.[a-zA-Z]+)");  // \w == [A-Za-z0-9_]
                    email = true;
                }
                else
                {
                    regex = new System.Text.RegularExpressions.Regex(@"^((?:\+|[0][0])[0-9]{1,15}$)");
                    email= false;
                }

                match = regex.Match(this.Locator);
                regexMatchReturn = match.get_Success();
```

```
                if(regexMatchReturn==false)
                {
                    if(email)
                    {
                        warning("The email address must contain @ character");
                    }
                    else
                    {
                        warning("The phone number must include optional + sign and maximum 15 digits");
                    }
                    return false;
                }
            }

            if(this.Type == LogisticsElectronicAddressMethodType::Email && this.IsPrimary == NoYes::Yes)
            {
                select firstonly logisticsElectronicAddress where logisticsElectronicAddress.Locator == this.Locator;

                if(logisticsElectronicAddress)
                {
                    warning("The email address is used by another customer");
                    return false;
                }
            }
        }
        return isValid;
    }
}
```

Clear table buffer:

axmReport.clear(); - ovime se prazni bafer tabele. Ne brise se sadrzaj iz baze opdataka

SQL

SQL server management studio:

T-SQL koristi: = , AND, OR, ' ', join TableName on
Transaction-SQL ne koristi: while, forupdate, &&, || , ==, " "

X++:
Umesto ON se koristi WHERE
Koristi se: count(id), group by, exists join(1. Tabela ima relaciju sa 2. I ako u 2. postoji record vrati mi ono iz 1. tabele), &&, ||

Primer upita:
select, from, where, join, from, where

```
while select salesTable
    where salesTable.SalesId == "012506"
    join salesLine
    where salesLine.SalesId == salesTable.SalesId
```

Upiti koje zelim da spojim:

```
while select VehiclePartId, VehiclePartQty from axmVehicleLines
    where axmVehicleLines.VehicleId == vehicle.VehicleId
{
    select firstonly forupdate VehiclePartQty from axmVehicleParts where axmVehicleLines.VehiclePartId == axmVehicleParts.VehiclePartId;
```

Spojeno u 1 u VS:

```
while select forupdate VehiclePartQty from axmVehicleParts
    join axmVehicleLines
    where axmVehicleLines.VehicleId == vehicle.VehicleId &&
          axmVehicleLines.VehiclePartId == axmVehicleParts.VehiclePartId
{
```

Spojeno u 1 u SQL:

```
select axmVehicleParts.VehiclePartId, axmVehicleParts.VehiclePartQty from axmVehicleParts
            join  axmVehicleLines
            on axmVehicleLines.VehicleId = 'VEH-00062' and
                  axmVehicleLines.VehiclePartId = axmVehicleParts.VehiclePartId
```

Spajanje upita sa if petljama u 1 upit:

```
while select * from axmVehicleTable join axmVehicleLines
    where axmVehicleLines.VehicleId == axmVehicleTable.VehicleId
{
    if(axmVehicleType == AXMVehicleTypeNone::None)
    {
        if(axmVehicleTable.VehicleAssembleDate > fromDate && axmVehicleTable.VehicleAssembleDate < toDate)
        {
            ttsbegin;
            axmReport.clear();
            axmReport.VehicleId = axmVehicleTable.VehicleId;
            axmReport.VehicleModel = axmVehicleTable.VehicleModel;
            axmReport.VehicleType = axmVehicleTable.VehicleType;
            axmReport.VehiclePartName = axmVehicleLines.VehiclePartName;
            axmReport.VehiclePartPrice = axmVehicleLines.VehiclePartPrice;
            axmReport.VehiclePartQty = axmVehicleLines.VehiclePartQty;
            axmReport.insert();
            ttscommit;
        }
    }
    else if(enum2Str(axmVehicleTable.VehicleType) == enum2Str(axmVehicleType) && axmVehicleTable.VehicleAssembleDate > fromDate &&
      axmVehicleTable.VehicleAssembleDate < toDate)
    {
        ttsbegin;
        axmReport.clear();
        axmReport.VehicleId = axmVehicleTable.VehicleId;
        axmReport.VehicleModel = axmVehicleTable.VehicleModel;
        axmReport.VehicleType = axmVehicleTable.VehicleType;
        axmReport.VehiclePartName = axmVehicleLines.VehiclePartName;
        axmReport.VehiclePartPrice = axmVehicleLines.VehiclePartPrice;
        axmReport.VehiclePartQty = axmVehicleLines.VehiclePartQty;
        axmReport.insert();
        ttscommit;
    }
}
```

```
while select * from axmVehicleTable join axmVehicleLines
    where axmVehicleLines.VehicleId == axmVehicleTable.VehicleId
    && axmVehicleTable.VehicleAssembleDate > fromDate
    && axmVehicleTable.VehicleAssembleDate < toDate
{
    if((axmVehicleType == AXMVehicleTypeNone::None) || enum2Str(axmVehicleTable.VehicleType) == enum2Str(axmVehicleType))
    {
        ttsbegin;
        axmReport.clear();
        axmReport.VehicleId = axmVehicleTable.VehicleId;
        axmReport.VehicleModel = axmVehicleTable.VehicleModel;
        axmReport.VehicleType = axmVehicleTable.VehicleType;
        axmReport.VehiclePartName = axmVehicleLines.VehiclePartName;
        axmReport.VehiclePartPrice = axmVehicleLines.VehiclePartPrice;
        axmReport.VehiclePartQty = axmVehicleLines.VehiclePartQty;
        axmReport.insert();
        ttscommit;
    }
}
```

Event Handlers

Insert operacija sa prikupljanjem podacima sa forme

```
[FormControlEventHandler(formControlStr(AXMProducts, NewProductButton), FormControlEventType::Clicked)]
1 reference
public static void NewProductButton_OnClicked(FormControl sender, FormControlEventArgs e)
{
    Dialog dialog = new Dialog("@AXMLabels:AXM923");
    // dialog.addFieldValue(extendedTypeStr(AXMProductNumber), axmProductNumber); Videce tek kad doda
    DialogField fieldName = dialog.addField(extendedTypeStr(AXMProductName));
    DialogField fieldType = dialog.addField(extendedTypeStr(AXMProductTypeEDT));
    DialogField fieldBrand = dialog.addField(extendedTypeStr(AXMProductBrand));
    DialogField fieldWarranty = dialog.addField(extendedTypeStr(AXMProductWarranty));
    DialogField fieldUnitPrice = dialog.addField(extendedTypeStr(UnitPrice));
    DialogField fieldColor = dialog.addField(extendedTypeStr(AXMProductColorEDT));
    DialogField fieldRGB = dialog.addField(extendedTypeStr(AXMIsRGB));

    dialog.run();

    if(dialog.closedOk())
    {
        AXMProductsTable axmProductsTable;

        AXMProductNumber axmProductNumber;
        NumberSequenceReference numberSequenceReference;

        numberSequenceReference = NumberSeqReference::findReference(extendedTypeNum(AXMProductNumber));
        if(numberSequenceReference)
        {
            ttsbegin;
            axmProductNumber = NumberSeq::newGetNum(numberSequenceReference).num();
            axmProductsTable.ProductId = axmProductNumber;
            axmProductsTable.ProductName = fieldName.value();
            axmProductsTable.ProductType = FieldType.value();
            axmProductsTable.ProductBrand = fieldBrand.value();
            axmProductsTable.Warranty = fieldWarranty.value();
            axmProductsTable.UnitPrice = fieldUnitPrice.value();
            axmProductsTable.ProductColor = fieldColor.value();
            axmProductsTable.IsRGB = fieldRGB.value();
            axmProductsTable.Insert();
            ttscommit;
        }
        info("@AXMLabels:AXM930" + axmProductNumber + "@AXMLabels:AXM929");
    }
```

Lookup

```
[FormControlEventHandler(formControlStr(AXMOrdersTable, AXMOrderLines_ProductId), FormControlEventType::Lookup)]
1 reference
public static void AXMOrderLines_ProductId_OnLookup(FormControl sender, FormControlEventArgs e)
{
    FormControl caller = sender as FormControl;
    FormRun form = caller.formRun();
    FormDataSource axmOrderLines_ds = form.dataSource(formDataSourceStr(AXMOrdersTable, AXMOrderLines)) as FormDataSource;
    AXMOrderLines axmOrderLinesCursor = axmOrderLines_ds.cursor();

    SysTableLookup sysTableLookup = SysTableLookup::newParameters(tableNum(AXMProductsTable), sender);

    // Add the lookup columns
    sysTableLookup.addLookupfield(fieldNum(AXMProductsTable, ProductId));
    sysTableLookup.addLookupfield(fieldNum(AXMProductsTable, ProductTypeEDT));
    sysTableLookup.addLookupfield(fieldNum(AXMProductsTable, Warranty));
    sysTableLookup.addLookupfield(fieldNum(AXMProductsTable, ProductColor));
    sysTableLookup.addLookupfield(fieldNum(AXMProductsTable, UnitPrice));

    // Create a query range for filtering by selected ProductType
    Query query = new Query();
    QueryBuildDataSource qbds = query.addDataSource(tableNum(AXMProductsTable));
    QueryBuildRange qbr = qbds.addRange(fieldNum(AXMProductsTable, ProductTypeEDT));
    qbr.value(queryValue(axmOrderLinesCursor.ProductType));

    // Set the modified query to the SysTableLookup
    sysTableLookup.parmQuery(query);

    // Run the lookup
    sysTableLookup.performFormLookup();

    // Cancel the call to super() to prevent the system from trying to show
    //     the lookup form twice and cause an error.
    FormControlCancelableSuperEventArgs cancelableSuperEventArgs = e as FormControlCancelableSuperEventArgs;
    cancelableSuperEventArgs.CancelSuperCall();
}
```